

HOW TO USE

이 자료 사용 방법

1. 수업·강의·자습에 그대로 쓰셔도 됩니다. 출처만 남겨 주세요.
2. PDF 는 화면으로 읽는 용도, PPTX 는 발표용입니다. PPTX 는 Pretendard 글꼴이 설치된 컴퓨터에서 가장 똑같이 보입니다.
3. 출처 표기 예: 이애본(드림아이티비즈), 「자료 제목」, 홍문원, useful-ai.dreamitbiz.com, CC BY-NC-ND 4.0
4. 자료 안의 프롬프트는 복사해 바로 쓸 수 있습니다. [대괄호]만 내 상황으로 바꾸세요.
5. 모든 자료와 영상 강의는 useful-ai.dreamitbiz.com 에서 다시 받을 수 있습니다.

저작권 · 이용 조건 (CC BY-NC-ND 4.0)

- 저작권 © 2026 이애본(드림아이티비즈 DreamIT Biz). 발행 홍문원(洪文院, 출판사 신고번호 제2026-000026호). 이애본이 2026년 대학·기관 강의 현장에서 쓴 강의안을 전자출판용으로 다시 묶은 것입니다. ISBN: 등록 후 기재.
- 이 자료는 크리에이티브 커먼즈 저작자표시-비영리-변경금지 4.0 국제(CC BY-NC-ND 4.0) 라이선스를 따릅니다. creativecommons.org/licenses/by-nc-nd/4.0/deed.ko
- 저작자 표시(BY): 출처를 밝히면 그대로 복사해 수업·강의·자습에 쓸 수 있습니다. 비영리(NC): 유료 강의·판매·광고 수익 목적에는 쓸 수 없습니다. 변경금지(ND): 내용을 고치거나 잘라 재조합해 배포할 수 없습니다.
- 자료 안의 외부 서비스 화면 캡처와 상표는 교육 목적 인용이며 각 서비스의 소유입니다. AI 생성 이미지는 생성 도구의 이용 약관을 따릅니다.
- 라이선스 범위 밖의 이용(영리·변형·2차 저작)은 먼저 문의해 주세요. aebon@dreamitbiz.com · 카카오톡 ID aebon

H01 · 프로그래밍 입문

파이썬 기초·응용·심화

설치부터 작은 프로젝트·웹 크롤링까지 — 처음 배우는 사람을 위한 파이썬

이애본 지음 · 홍문원(洪文院) 발행

CC BY-NC-ND 4.0 · useful-ai.dreamitbiz.com

이 강의안의 순서

1	개발 환경 준비	준비	5	파일과 예외 처리	응용
2	기초 문법 — 변수·연산·조건·반복	기초	6	프로젝트 3종 — 계산기·숫자 맞추기·타이머	실습
3	자료구조 — 리스트·튜플·딕셔너리·세트	기초	7	웹 크롤링 맛보기	심화
4	함수와 모듈	응용	8	점검과 다음 학습	정리

이 강의안을 마치면

목표 ①	파이썬과 가상환경을 설치하고 코드를 실행한다
목표 ②	변수·조건·반복·자료구조로 작은 문제를 푼다
목표 ③	함수·모듈로 코드를 나누고 다시 쓴다
목표 ④	파일을 읽고 쓰며, 예외로 프로그램을 지킨다
목표 ⑤	규칙을 지켜 웹 페이지에서 데이터를 모은다

결과물 — 직접 실행해 본 프로젝트 3개와 수집한 CSV 파일 하나

시작 전에 갖출 것

컴퓨터

01

Windows 또는 macOS
설치 권한이 있는 계정

프로그램

02

Python(3.12 이상) · VS Code
— PART 1 에서 함께 설치

대안

03

설치가 어려우면 Google Colab
— 구글 계정과 브라우저만

마음가짐

04

눈으로 읽지 말고 직접 쳐 본다
— 오류 메시지도 교재다

PART 1

개발 환경 준비

파이썬을 설치하고, 프로젝트마다 가상환경을 만들고, VS Code 에서 첫 프로그램을 실행합니다

프로그래밍 언어란

01

컴퓨터의 말

컴퓨터는 0과 1로 된 명령만
직접 알아듣는다

02

통역 규칙

사람이 읽는 글로 쓰면
컴퓨터가 실행할 형태로 바뀐다

03

비유 — 레시피

순서대로 정확히 적은 지시문
한 줄만 틀려도 결과가 달라진다

같은 일, 다른 길이 — 파이썬과 자바

```
1 # 파이썬 — 한 줄이면 된다
2 print("Hello, World!")
3
4 # [실행 결과]
5 # Hello, World!
6
7 // 자바 — 클래스와 main 메서드 안에 써야 한다
8 public class Hello {
9     public static void main(String[] args) {
10         System.out.println("Hello, World!");
11     }
12 }
```

두 코드 모두 실행해 같은 결과를 확인(파이썬 3.12 · 자바 21) — 파이썬은 군더더기가 적어 입문에 알맞다

특징

파이썬의 강점과 약점

01 읽기 쉬운 문법

들여쓰기로 구조가 드러나
처음 배우는 사람도 읽고 쓰기 쉽다

02 넓은 쓰임

데이터 분석 · AI · 웹 · 업무 자동화
한 언어로 여러 분야를 다룬다

03 풍부한 라이브러리

pip 명령 한 줄로
남이 만든 도구를 가져다 쓴다

04 약점 — 실행 속도

C·자바보다 대체로 느리다
무거운 계산은 C로 만든 라이브러리에 맡긴다

python.org 에서 파이썬 설치

- 1 python.org 접속**
Downloads 에서 내 운영체제용 최신 안정판(3.12 이상)을 받는다
- 2 Windows**
설치 화면에 「Add python.exe to PATH」가 보이면 체크한다
- 3 macOS**
내려받은 .pkg 를 열어 안내대로 설치한다
- 4 확인**
터미널을 새로 열고 버전을 확인한다(다음 장)

기준

2026년 10월 기준
3.12~3.14 실행 확인

버전이 나오면 성공

```
1 # Windows — 명령 프롬프트 또는 PowerShell
2 py --version
3 python --version
4
5 # macOS — 터미널(macOS 는 python3 라는 이름을 쓴다)
6 python3 --version
7
8 # [실행 결과 예 — 숫자는 설치한 판에 따라 다르다]
9 # Python 3.12.15
```

「명령을 찾을 수 없다」가 나오면 PATH 문제 — 이 파트 끝의 오류 표를 본다

venv — 프로젝트마다 따로 쓰는 필통

```
1 # 1) 프로젝트 폴더에서 가상환경 만들기 → .venv 폴더가 생긴다
2 python -m venv .venv          # macOS 는 python3
3
4 # 2) 켜기
5 .venv\Scripts\activate       # Windows
6 source .venv/bin/activate    # macOS
7
8 # 3) 켜지면 프롬프트 앞에 (.venv) 가 붙는다
9 (.venv) $ python --version
10
11 # 4) 끄기
12 deactivate
```

컨 상태에서 설치한 라이브러리는 이 폴더 안에만 들어간다 — 프로젝트끼리 섞이지 않고, .venv 를 지우면 깨끗이 정리된다

pip

라이브러리 설치 — pip

```
1 pip install requests          # 설치
2 pip install requests beautifulsoup4 # 여러 개 한 번에
3 pip list                      # 설치된 목록
4 pip uninstall requests        # 지우기
5
6 # 설치 목록을 파일로 남기고, 다른 컴퓨터에서 그대로 재현
7 pip freeze > requirements.txt
8 pip install -r requirements.txt
```

가상환경을 켜 뒤에 설치한다 — 켜지 않으면 컴퓨터 전체 파이썬에 들어가 다른 프로젝트와 섞인다

VS Code 준비

1

VS Code 설치

code.visualstudio.com 에서 내려받아 설치

2

Python 확장 설치

왼쪽 확장(Extensions)에서 「Python」(Microsoft) 검색 → 설치

3

폴더 열기

File → Open Folder 로 프로젝트 폴더를 연다

4

인터프리터 고르기

Ctrl+Shift+P(macOS ⌘+Shift+P) → Python: Select Interpreter → .venv

확인

상태 표시줄에

.venv 가 보이면 끝

첫 실행

첫 프로그램 — hello.py

```
1 # hello.py
2 print("안녕하세요, 파이썬!")
3 print(2 + 3)
4 print("2 + 3 =", 2 + 3)
5
6 # [실행 결과]
7 # 안녕하세요, 파이썬!
8 # 5
9 # 2 + 3 = 5
```

실행 — 편집기 오른쪽 위 실행 버튼, 또는 터미널에서 `python hello.py`(macOS 는 `python3`)

다른 길

설치가 어렵다면 — Google Colab

VS Code + venv

내 컴퓨터에서

- 파일·폴더로 프로젝트를 관리
- 인터넷 없이도 실행
- 이 강의안의 기본 환경

Google Colab

colab.research.google.com

VS

- 구글 계정만 있으면 설치 없이 시작
- 셀 단위로 실행하는 노트북(Jupyter 형식)
- 세션이 끝나면 설치한 것이 사라짐 — !pip install 로 다시

Anaconda 는?

01

무엇인가

파이썬 + 데이터 과학 패키지를
한데 묶은 배포판(conda 명령)

02

이 강의안에서는

python.org + venv + pip 를 쓴다
— 구조가 단순하고 가볍다

03

주의

두 방식을 섞으면 어느 파이썬이
도는지 헷갈린다 · 기관은 약관 확인

설치에서 자주 막히는 곳

증상	원인	해결
python 명령을 찾을 수 없음	PATH 미등록	PATH 추가를 체크해 다시 설치 · Windows 는 py
macOS 에 python 이 없음	이름이 python3	python3 · pip3 로 실행
PowerShell 이 activate 를 거부	실행 정책	Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
설치했는데 import 가 안 됨	다른 파이썬에 설치	.venv 를 켜고 설치 · VS Code 도 .venv 선택

오류 문구를 그대로 검색창에 붙여 넣으면 대개 같은 일을 겪은 사람의 답이 나온다

환경 준비 점검

- ① 터미널에서 파이썬 버전이 나온다
- ② 프로젝트 폴더에 `.venv` 를 만들고 켤 수 있다
- ③ VS Code 에서 `.venv` 인터프리터를 골랐다
- ④ `hello.py` 를 실행해 출력을 봤다

넷 다 되면 PART 2 로 — 막히면 Colab 으로 먼저 진행해도 된다

PART 2

기초 문법

값을 담고(변수), 계산하고(연산), 고르고(조건), 되풀이(반복)합니다

변수 — 이름표 붙은 상자

01

상자

값을 담아 두는 곳
— 숫자·글자·참거짓

02

이름표

상자를 부르는 이름
— 뜻이 드러나게 짓는다

03

= 의 뜻

「같다」가 아니라
「오른쪽 값을 왼쪽에 넣어라」

변수

변수 만들고 바꾸기

```
1 name = "민지"
2 age = 20
3 print(name, age)
4
5 age = age + 1      # 오른쪽을 먼저 계산해 다시 넣는다
6 print(age)
7
8 my_score = 90      # 이름: 영문 · 숫자 · 밑줄, 숫자로 시작 불가
9 print(my_score)
10
11 # [실행 결과]
12 # 민지 20
13 # 21
14 # 90
```

기본 자료형 네 가지

자료형	이름	예	쓰임
int	정수	10, -3, 0	개수 · 나이
float	실수	3.14, 0.5	키 · 평균
str	문자열	"안녕", '파이썬'	이름 · 문장
bool	불리언	True, False	참 · 거짓 판단

문자열은 큰따옴표와 작은따옴표 둘 다 된다 — 시작과 끝만 같으면 된다

type() 으로 확인하고 형 변환하기

```
1 print(type(10))
2 print(type(3.14))
3 print(type("안녕"))
4 print(type(True))
5 # 형 변환 — 다른 자료형으로 바꾸기
6 print(int("42") + 1)
7 print(float("3.5") * 2)
8 print(str(100) + "점")
9 print(int(3.9))      # 소수점 아래를 버린다
10
11 # [실행 결과]
12 # <class 'int'>
13 # <class 'float'>
14 # <class 'str'>
15 # <class 'bool'>
16 # 43
17 # 7.0
18 # 100점
19 # 3
```

산술 연산자

```
1 a, b = 17, 5
2 print(a + b)    # 더하기
3 print(a - b)    # 빼기
4 print(a * b)    # 곱하기
5 print(a / b)    # 나누기 — 결과는 늘 실수
6 print(a // b)   # 몫
7 print(a % b)    # 나머지
8 print(a ** 2)   # 거듭제곱
9
10 # [실행 결과]
11 # 22
12 # 12
13 # 85
14 # 3.4
15 # 3
16 # 2
17 # 289
```

비유 — 사탕 17개를 5명에게 나누면 한 명당 3개(몫 //), 2개가 남는다(나머지 %)

문자열 — 번호로 꺼내고 자르기

```
1 s = "Python"
2 print(s[0], s[-1])    # 첫 글자, 마지막 글자
3 print(s[0:3])         # 0번부터 3번 앞까지
4 print(len(s))         # 글자 수
5 print("파이" + "썬")  # 이어 붙이기
6 print("ha" * 3)        # 되풀이
7 print("y" in s)        # 들어 있나?
8
9 # [실행 결과]
10 # P n
11 # Pyt
12 # 6
13 # 파이썬
14 # hahaha
15 # True
```

번호(인덱스)는 0부터 센다 — 음수는 뒤에서부터. [시작:끝] 은 끝 번호를 포함하지 않는다

문자열 메서드와 f-string

```
1 text = " Hello, Python "
2 print(text.strip())          # 양끝 공백 제거
3 print(text.strip().upper())  # 대문자로
4 print(text.strip().replace("Python", "World"))
5 print("a,b,c".split(","))    # 쉼표로 나누기
6
7 name, score = "민지", 92.456
8 print(f"{name}님의 점수는 {score}점")
9 print(f"소수 둘째 자리: {score:.2f}")
10 print(f"천 단위 쉼표: {1234567:,}")
11
12 # [실행 결과]
13 # Hello, Python
14 # HELLO, PYTHON
15 # Hello, World
16 # ['a', 'b', 'c']
17 # 민지님의 점수는 92.456점
18 # 소수 둘째 자리: 92.46
19 # 천 단위 쉼표: 1,234,567
```

input() — 키보드로 입력받기

```
1 name = input("이름을 입력하세요: ")
2 age = int(input("나이를 입력하세요: ")) # 글자 → 정수
3 print(f"{name}님은 내년엔 {age + 1}살입니다.")
4
5 # [실행 예 — 민지, 20 을 입력]
6 # 이름을 입력하세요: 민지
7 # 나이를 입력하세요: 20
8 # 민지님은 내년엔 21살입니다.
```

input() 이 돌려주는 값은 언제나 문자열 — 계산하려면 int() · float() 로 바꾼다

흔한 오류 ① 문자열 + 숫자

```
1 age = 20
2 print("나이:", age)           # 쉼표로 나열 — 된다
3 print(f"나이: {age}")         # f-string — 된다
4 print("나이: " + str(age))    # 문자열로 바꿔 붙이기 — 된다
5 print("나이: " + age)         # 문자열 + 정수 — 오류!
6
7 # [실행 결과 — 오류]
8 # 나이: 20
9 # 나이: 20
10 # 나이: 20
11 # TypeError: can only concatenate str (not "int") to str
```

오류 메시지는 맨 아래 줄부터 읽는다 — 「TypeError: 문자열(str)에는 문자열만 이어 붙일 수 있다」

비교 연산자와 논리 연산자

```
1  x = 7
2  print(x > 5)           # 크다
3  print(x == 7)          # 같다(= 하나는 대입)
4  print(x != 7)          # 다르다
5  print(x > 5 and x < 10) # 둘 다 참이어야 참
6  print(x < 5 or x > 6)  # 하나만 참이어도 참
7  print(not x > 5)       # 참 · 거짓 뒤집기
8  print(5 < x < 10)      # 이어 쓰기도 된다
9
10 # [실행 결과]
11 # True
12 # True
13 # False
14 # True
15 # True
16 # False
17 # True
```

if · elif · else — 갈림길 만들기

```
1 score = 85
2
3 if score >= 90:
4     grade = "A"
5 elif score >= 80:
6     grade = "B"
7 elif score >= 70:
8     grade = "C"
9 else:
10    grade = "F"
11
12 print(f"{score}점은 {grade}학점")
13
14 # [실행 결과]
15 # 85점은 B학점
```

위에서부터 검사해 처음 참인 곳 하나만 실행한다 — 그래서 큰 기준부터 쓴다

흔한 오류 ② 들여쓰기와 = · ==

```
1 score = 85
2 if score >= 80:
3     print("합격")
4 # → IndentationError: expected an indented block after 'if' statement on line 2
5
6 score = 85
7 if score = 85:
8     print("딱 85점")
9 # → SyntaxError: invalid syntax. Maybe you meant '==' or ':=' instead of '='?
```

콜론(:) 다음 줄은 4칸 들여쓴다 · 같은 블록은 같은 칸 수로 · 비교는 ==, 대입은 =

for 와 range — 정해진 횟수만큼

```
1  for i in range(5):           # 0, 1, 2, 3, 4
2      print(i, end=" ")
3  print()
4
5  for i in range(1, 11, 3):     # 1부터 10까지 3씩 건너뛰며
6      print(i, end=" ")
7  print()
8
9  total = 0
10 for n in range(1, 101):
11     total += n                # total = total + n 의 줄임
12 print("1부터 100까지 합:", total)
13
14 # [실행 결과]
15 # 0 1 2 3 4
16 # 1 4 7 10
17 # 1부터 100까지 합: 5050
```

range(끝) · range(시작, 끝) · range(시작, 끝, 간격) — 끝 숫자는 포함하지 않는다

while — 조건이 참인 동안

```
1 count = 3
2 while count > 0:
3     print(f"{count}...")
4     count -= 1          # 빼먹으면 끝나지 않는다
5 print("발사!")
6
7 # [실행 결과]
8 # 3...
9 # 2...
10 # 1...
11 # 발사!
```

몇 번 돌지 미리 모를 때 while 을 쓴다 — 언젠가 조건이 거짓이 되도록 값을 바꿔 준다

break 와 continue

```
1  for n in range(1, 11):
2      if n % 2 == 0:
3          continue          # 짝수는 건너뛰고 다음 바퀴로
4      if n > 7:
5          break              # 7보다 크면 반복을 끝낸다
6      print(n, end=" ")
7  print()
8  print("반복 끝")
9
10 # [실행 결과]
11 # 1 3 5 7
12 # 반복 끝
```

흔한 오류 ③ 끝나지 않는 반복

```
1  n = 1
2  while n <= 3:
3      print(n)
4      # n += 1 을 빠뜨렸다 → n 은 계속 1
5
6  # [실행 결과]
7  # 1
8  # 1
9  # 1
10 # ... (끝나지 않는다 — Ctrl+C 로 멈춘다)
```

화면이 멈추고 같은 줄만 쏟아지면 무한 반복 — Ctrl+C 로 멈추고 조건이 바뀌는지 확인한다

몫과 나머지

`print(17 // 5, 17 % 5)`의 출력은?

① 3.4 0

② 3 2

③ 3 3.4

④ 2 3

확인 문제

정답과 해설

정답

② 3 2

해설

// 는 몫, % 는 나머지입니다. $17 = 5 \times 3 + 2$ 이므로 몫 3, 나머지 2 — print 는 십표로 나열한 값을 빈칸 하나로 띄워 출력합니다

기초 문법 점검

- ① 변수에 값을 넣고 print 로 확인한다
- ② int · float · str · bool 과 형 변환을 구분한다
- ③ if · elif · else 로 갈림길을 만든다
- ④ for · while 로 반복하고 break 로 멈춘다
- ⑤ 오류 메시지 마지막 줄을 읽고 원인을 찾는다

PART 3

자료구조

값 여러 개를 한데 담는 그릇 네 가지 — 리스트 · 튜플 · 딕셔너리 · 세트

네 가지 그릇

리스트 []

01

번호 붙은 사물함

순서 있음 · 넣고 빼고 바꿀 수 있다

튜플 ()

02

봉인된 상자

순서 있음 · 만든 뒤에는 바꿀 수 없다

딕셔너리 { 키: 값 }

03

국어사전

단어(키)로 뜻(값)을 바로 찾는다

세트 { }

04

스티커 도감

같은 것은 한 번만 들어간다

리스트 — 번호로 꺼내고 바꾸기

```
1  fruits = ["사과", "바나나", "포도", "딸기"]
2  print(fruits[0])      # 첫째 — 번호는 0부터
3  print(fruits[-1])     # 끝에서 첫째
4  print(fruits[1:3])    # 1번부터 3번 앞까지
5  print(len(fruits))
6
7  fruits[1] = "망고"    # 바꾸기
8  print(fruits)
9
10 # [실행 결과]
11 # 사과
12 # 딸기
13 # ['바나나', '포도']
14 # 4
15 # ['사과', '망고', '포도', '딸기']
```

리스트 메서드

```
1  nums = [3, 1, 4]
2  nums.append(1)      # 끝에 추가
3  nums.insert(0, 9)   # 0번 자리에 끼우기
4  print(nums)
5  nums.remove(1)      # 값 1 을 앞에서부터 하나 지우기
6  last = nums.pop()   # 끝 값을 꺼내며 지우기
7  print(nums, last)
8  nums.sort()         # 오름차순 정렬
9  print(nums)
10 print(4 in nums, nums.count(4))
11
12 # [실행 결과]
13 # [9, 3, 1, 4, 1]
14 # [9, 3, 4] 1
15 # [3, 4, 9]
16 # True 1
```

흔한 오류 ④ 없는 번호

```
1 fruits = ["사과", "바나나", "포도"]
2 print(len(fruits))
3 print(fruits[3])      # 번호는 0, 1, 2 까지만 있다
4
5 # [실행 결과 — 오류]
6 # 3
7 # IndexError: list index out of range
```

항목이 3개면 번호는 0~2 — 마지막은 len(리스트) - 1 또는 [-1] 로 꺼낸다

리스트와 반복 — 합계·평균·거르기

```
1 scores = [88, 92, 75, 64, 95]
2
3 print("합계:", sum(scores))
4 print("평균:", sum(scores) / len(scores))
5 print("최고:", max(scores), "최저:", min(scores))
6
7 passed = []
8 for s in scores:
9     if s >= 80:
10         passed.append(s)
11 print("80점 이상:", passed)
12 print([s for s in scores if s >= 80])    # 같은 일을 한 줄로(컴프리헨션)
13
14 # [실행 결과]
15 # 합계: 414
16 # 평균: 82.8
17 # 최고: 95 최저: 64
18 # 80점 이상: [88, 92, 95]
19 # [88, 92, 95]
```

튜플 — 바꿀 수 없는 묶음

```
1 point = (3, 5)
2 x, y = point      # 언패킹 — 하나씩 나눠 담기
3 print(x, y)
4
5 a, b = 1, 2
6 a, b = b, a      # 두 값 바꾸기도 한 줄
7 print(a, b)
8
9 point[0] = 10     # 튜플은 바꿀 수 없다
10
11 # [실행 결과 — 오류]
12 # 3 5
13 # 2 1
14 # TypeError: 'tuple' object does not support item assignment
```

좌표·날짜처럼 바뀌면 안 되는 값을 묶을 때 쓴다 — 바꾸려 하면 `TypeError`

딕셔너리 — 키로 찾기

```
1 student = {"이름": "민지", "학년": 2}
2 print(student["이름"])
3
4 student["전공"] = "컴퓨터"      # 없는 키 → 추가
5 student["학년"] = 3             # 있는 키 → 수정
6 del student["전공"]             # 지우기
7 print(student)
8
9 print(student.get("나이"))      # 없으면 None
10 print(student.get("나이", 0))  # 없을 때 쓸 기본값
11 print("이름" in student)
12
13 # [실행 결과]
14 # 민지
15 # {'이름': '민지', '학년': 3}
16 # None
17 # 0
18 # True
```

딕셔너리 순회 — items()

```
1 stock = {"사과": 5, "바나나": 0, "포도": 12}
2
3 for name, qty in stock.items():
4     if qty == 0:
5         print(f"{name}: 품절")
6     else:
7         print(f"{name}: {qty}개")
8
9 print(list(stock.keys()))
10 print(sum(stock.values()))
11
12 # [실행 결과]
13 # 사과: 5개
14 # 바나나: 품절
15 # 포도: 12개
16 # ['사과', '바나나', '포도']
17 # 17
```

흔한 오류 ⑤ 없는 키

```
1 stock = {"사과": 5}
2 print(stock.get("배", 0))    # 안전한 조회
3 print(stock["배"])           # 없는 키 → 오류
4
5 # [실행 결과 — 오류]
6 # 0
7 # KeyError: '배'
```

있는지 모르는 키는 `get(키, 기본값)` 으로 꺼내거나 `if 키 in 딕셔너리:` 로 먼저 확인한다

세트 — 중복 없애기와 집합 연산

```
1 visits = ["민지", "준호", "민지", "서연", "준호"]
2 people = set(visits)          # 중복 제거
3 print(len(people), sorted(people))
4
5 a = {"파이썬", "자바", "C"}
6 b = {"파이썬", "자바스크립트"}
7 print(a & b)                  # 교집합
8 print(sorted(a | b))         # 합집합
9 print(sorted(a - b))         # 차집합
10
11 # [실행 결과]
12 # 3 ['민지', '서연', '준호']
13 # {'파이썬'}
14 # ['C', '자바', '자바스크립트', '파이썬']
15 # ['C', '자바']
```

세트는 순서가 없어 출력 순서가 실행마다 달라질 수 있다 — 그래서 `sorted()` 로 정렬해 보여 준다

네 그릇 한눈에

그릇	모양	순서	바꾸기	중복
리스트	[1, 2]	있음	가능	허용
튜플	(1, 2)	있음	불가	허용
딕셔너리	{"a": 1}	넣은 순서 유지	가능	키는 불가
세트	{1, 2}	없음	가능	불가

고르는 법 — 차례가 중요하면 리스트, 이름으로 찾으면 딕셔너리, 중복을 없애려면 세트

PART 4

함수와 모듈

자주 쓰는 코드를 묶어 이름을 붙이고(함수), 파일로 나눠 가져다 씁니다(모듈)

함수 — 자판기

01

넣는 것

매개변수
— 동전과 버튼

02

하는 일

함수 본문
— 기계 안의 동작

03

나오는 것

반환값(return)
— 떨어지는 음료

def 로 정의하고 호출하기

```
1  def greet(name):           # 정의 — 아직 실행되지 않는다
2      return f"안녕하세요, {name}님!"
3
4  msg = greet("민지")        # 호출 — 이때 실행된다
5  print(msg)
6  print(greet("준호"))
7
8  def area(width, height):
9      return width * height
10
11 print(area(3, 4))
12
13 # [실행 결과]
14 # 안녕하세요, 민지님!
15 # 안녕하세요, 준호님!
16 # 12
```

기본값과 키워드 인자

```
1 def order(menu, size="보통", count=1):
2     return f"{menu} {size} {count}잔"
3
4 print(order("라떼"))           # 기본값 사용
5 print(order("라떼", "큰"))     # 순서대로
6 print(order("라떼", count=3))  # 이름으로 골라 넘기기
7 print(order(count=2, menu="녹차")) # 이름이면 순서 무관
8
9 # [실행 결과]
10 # 라떼 보통 1잔
11 # 라떼 큰 1잔
12 # 라떼 보통 3잔
13 # 녹차 보통 2잔
```

print 와 return 은 다르다

```
1 def add_print(a, b):
2     print(a + b)          # 화면에 보여 주기만 한다
3
4 def add_return(a, b):
5     return a + b          # 값을 돌려준다
6
7 x = add_print(2, 3)
8 y = add_return(2, 3)
9 print("x =", x)           # 돌려준 값이 없으면 None
10 print("y =", y)
11 print(add_return(2, 3) * 10) # 돌려받은 값은 계속 쓸 수 있다
12
13 # [실행 결과]
14 # 5
15 # x = None
16 # y = 5
17 # 50
```

print 는 사람에게 보여 주기, return 은 프로그램에 돌려주기 — 계산 결과를 다시 쓰려면 return

함수

여러 값 돌려주기와 변수의 범위

```
1 def min_max(nums):
2     return min(nums), max(nums)    # 두 값을 튜플로
3
4 lo, hi = min_max([4, 9, 1, 7])
5 print(lo, hi)
6
7 def make_total():
8     total = 100                    # 함수 안에서 만든 지역 변수
9     return total
10
11 make_total()
12 print(total)                      # 함수 밖에서는 보이지 않는다
13
14 # [실행 결과 — 오류]
15 # 1 9
16 # NameError: name 'total' is not defined
```

함수 안에서 만든 변수는 함수가 끝나면 사라진다 — 밖에서 쓰려면 return 으로 돌려받는다

람다 — 이름 없는 한 줄 함수

```
1 square = lambda x: x * x          # 이름 없는 한 줄 함수
2 print(square(5))
3
4 words = ["banana", "kiwi", "apple"]
5 print(sorted(words))              # 알파벳순
6 print(sorted(words, key=len))     # 길이순
7 print(sorted(words, key=lambda w: w[-1])) # 끝 글자순
8
9 students = [("민지", 92), ("준호", 78), ("서연", 85)]
10 best = max(students, key=lambda s: s[1])
11 print("1등:", best[0])
12
13 # [실행 결과]
14 # 25
15 # ['apple', 'banana', 'kiwi']
16 # ['kiwi', 'apple', 'banana']
17 # ['banana', 'apple', 'kiwi']
18 # 1등: 민지
```

sorted · max 의 key 에 「무엇을 기준으로 비교할지」를 넘길 때 가장 많이 쓴다

자주 쓰는 내장 함수

함수	하는 일	예 → 결과
len	길이(개수)	len("abc") → 3
sum · max · min	합 · 최대 · 최소	max([3, 8, 2]) → 8
sorted	정렬한 새 리스트	sorted([3, 1, 2]) → [1, 2, 3]
round	반올림	round(3.14159, 2) → 3.14
abs	절댓값	abs(-7) → 7
range	정수 범위	list(range(3)) → [0, 1, 2]
isinstance	자료형 확인	isinstance(3, int) → True

내장 함수는 import 없이 바로 쓴다 — 전체 목록은 파이썬 공식 문서 「Built-in Functions」

enumerate 와 zip

```
1 menu = ["김밥", "라면", "떡볶이"]
2 for i, item in enumerate(menu, start=1):    # 번호 붙이기
3     print(i, item)
4
5 prices = [3000, 4500, 5000]
6 for item, price in zip(menu, prices):      # 짝 맞추기
7     print(f"{item}: {price:,}원")
8
9 print(dict(zip(menu, prices)))
10
11 # [실행 결과]
12 # 1 김밥
13 # 2 라면
14 # 3 떡볶이
15 # 김밥: 3,000원
16 # 라면: 4,500원
17 # 떡볶이: 5,000원
18 # {'김밥': 3000, '라면': 4500, '떡볶이': 5000}
```

표준 라이브러리와 import 세 가지

```

1  import math                # ① 모듈 통째로 — math.이름
2  from datetime import date  # ② 필요한 것만 — 이름만으로
3  import random as rd        # ③ 별명 붙이기
4
5  print(math.sqrt(16), math.floor(3.7))
6  print(round(math.pi, 3))
7
8  rd.seed(1)                 # 같은 결과를 다시 보려고 씨앗 고정
9  print(rd.randint(1, 6))     # 주사위 1~6
10 print(rd.choice(["가위", "바위", "보"]))
11
12 print(date(2026, 10, 9).weekday()) # 0=월요일 ... 6=일요일
13
14 # [실행 결과]
15 # 4.0 3
16 # 3.142
17 # 2
18 # 보
19 # 4

```

내 모듈 만들기과 `__name__`

```

1  # 파일 ① mytools.py
2  def add(a, b):
3      return a + b
4
5  if __name__ == "__main__":      # 이 파일을 직접 실행할 때만
6      print("시험:", add(1, 1))
7
8  # 파일 ② main.py — 같은 폴더
9  import mytools                  # 가져올 때는 시험 코드가 돌지 않는다
10 from mytools import add
11
12 print(mytools.add(10, 20))
13 print(add(2, 3))
14
15 # [python mytools.py 결과]  시험: 2
16 # [python main.py 결과]    30 / 5

```

파일 하나가 모듈이다 — `__name__` 은 직접 실행하면 "`__main__`", import 되면 모듈 이름이 된다

클래스 맛보기 — 붕어빵 틀

```
1 class Dog:                # 붕어빵 틀
2     def __init__(self, name): # 만들 때 한 번 실행(초기화)
3         self.name = name      # 이 붕어빵의 속성
4         self.tricks = []
5     def learn(self, trick):    # 메서드 — 이 객체가 하는 일
6         self.tricks.append(trick)
7     def show(self):
8         return f"{self.name}: {' '.join(self.tricks)}"
9
10 a = Dog("초코")            # 틀로 찍어 낸 객체 둘
11 b = Dog("보리")
12 a.learn("앉아"); a.learn("손")
13 b.learn("엎드려")
14 print(a.show())
15 print(b.show())
16
17 # [실행 결과]
18 # 초코: 앉아, 손
19 # 보리: 엎드려
```

클래스는 틀, 객체는 틀로 찍은 붕어빵 — 객체마다 자기 속성(이름·재주)을 따로 가진다

흔한 오류 ⑥ 모듈이 없다

```
1 from bs4 import BeautifulSoup      # 설치하지 않은 라이브러리
2
3 # [실행 결과 — 오류]
4 # ModuleNotFoundError: No module named 'bs4'
5
6 # 해결 — 가상환경을 켜 뒤 설치(설치 이름과 import 이름이 다르다)
7 # pip install beautifulsoup4
```

설치했는데도 같은 오류면 VS Code 가 다른 파이썬을 쓰는 중 — 인터프리터가 .venv 인지 확인한다

자료구조·함수·모듈 점검

- ① 리스트·튜플·딕셔너리·세트를 골라 쓴다
- ② IndexError · KeyError 의 뜻을 안다
- ③ 매개변수와 return 이 있는 함수를 만든다
- ④ import 세 방식과 `__name__` 을 설명한다
- ⑤ pip 이름과 import 이름이 다를 수 있음을 안다

PART 5

파일과 예외 처리

프로그램이 끝나도 남도록 파일에 쓰고, 문제가 생겨도 멈추지 않도록 예외를 처리합니다

파일 여는 방식(모드)

01

"r" 읽기

있는 공책을 펼쳐 읽기만
— 기본값

02

"w" 쓰기

새 공책 — 같은 이름이 있으면
내용을 지우고 처음부터

03

"a" 이어 쓰기

공책 마지막 줄 뒤에
덧붙인다

with open — 쓰고, 이어 쓰고, 읽기

```
1 # with — 블록이 끝나면 파일을 자동으로 닫는다
2 with open("memo.txt", "w", encoding="utf-8") as f:
3     f.write("첫째 줄\n")
4     f.write("둘째 줄\n")
5
6 with open("memo.txt", "a", encoding="utf-8") as f:
7     f.write("덧붙인 줄\n")
8
9 with open("memo.txt", encoding="utf-8") as f:
10     print(f.read())
11
12 # [실행 결과]
13 # 첫째 줄
14 # 둘째 줄
15 # 덧붙인 줄
```

한 줄씩 읽기

```
1 with open("memo.txt", encoding="utf-8") as f:
2     for i, line in enumerate(f, start=1):
3         print(i, line.strip())      # 줄 끝의 \n 을 떼고 출력
4
5 with open("memo.txt", encoding="utf-8") as f:
6     lines = f.readlines()          # 줄들을 리스트로
7 print(len(lines), "줄")
8
9 # [실행 결과]
10 # 1 첫째 줄
11 # 2 둘째 줄
12 # 3 덧붙인 줄
13 # 3 줄
```

| encoding="utf-8" 을 늘 붙인다 — 빼면 Windows 에서 한글이 깨지거나 UnicodeDecodeError 가 난다

CSV 파일 — 표 데이터 저장하기

```
1 import csv
2
3 rows = [{"이름", "점수"}, {"민지", 92}, {"준호", 78}]
4 with open("scores.csv", "w", newline="", encoding="utf-8") as f:
5     csv.writer(f).writerows(rows)
6
7 with open("scores.csv", newline="", encoding="utf-8") as f:
8     for row in csv.DictReader(f):          # 첫 줄을 열 이름으로
9         print(row["이름"], int(row["점수"]) + 5)
10
11 # [실행 결과]
12 # 민지 97
13 # 준호 83
```

엑셀에서 한글이 깨지면 `encoding="utf-8-sig"` 로 저장 · 표가 커지면 `pandas` 가 편하다(다음 학습)

예외 — 비상구

01

예외

실행 중에 생긴 문제
— 0으로 나누기, 없는 파일...

02

그대로 두면

프로그램이 그 자리에서
멈춘다

03

try · except

문제가 생기면 준비해 둔 길
(비상구)로 돌린다

try · except — 예외 잡기

```
1 def safe_divide(a, b):
2     try:
3         return a / b
4     except ZeroDivisionError:
5         return "0으로 나눌 수 없습니다"
6 print(safe_divide(10, 4))
7 print(safe_divide(10, 0))
8
9 for text in ["42", "사십이"]:
10     try:
11         print(int(text) * 2)
12     except ValueError:
13         print(f"'{text}' 는 숫자가 아닙니다")
14
15 # [실행 결과]
16 # 2.5
17 # 0으로 나눌 수 없습니다
18 # 84
19 # '사십이' 는 숫자가 아닙니다
```

예외

else 와 finally

```
1 def read_file(path):
2     try:
3         f = open(path, encoding="utf-8")
4     except FileNotFoundError:
5         print(f"{path}: 파일이 없습니다")
6     else:
7         # 오류가 없을 때만
8         print(f.read().strip())
9         f.close()
10    finally:
11        # 오류가 있든 없든 늘
12        print("확인 끝")
13
14 read_file("memo.txt")
15 read_file("nofile.txt")
16
17 # [실행 결과]
18 # 안녕하세요
19 # 확인 끝
20 # nofile.txt: 파일이 없습니다
21 # 확인 끝
```

자주 만나는 예외

예외	언제	예
NameError	없는 이름을 썼다	<code>print(totl)</code>
TypeError	자료형이 맞지 않는다	<code>"나이: " + 20</code>
ValueError	값의 모양이 틀렸다	<code>int("사십")</code>
IndexError	없는 번호	<code>[1, 2][5]</code>
KeyError	없는 키	<code>{"a": 1}["b"]</code>
ZeroDivisionError	0으로 나눴다	<code>10 / 0</code>
FileNotFoundError	없는 파일을 열었다	<code>open("없음.txt")</code>
ModuleNotFoundError	설치 안 된 모듈	<code>from bs4 import ...</code>

모든 예를 실제로 실행해 표의 예외가 나는 것을 확인했다 — `except` 에는 잡을 예외 이름을 구체적으로 쓴다

파일·예외 점검

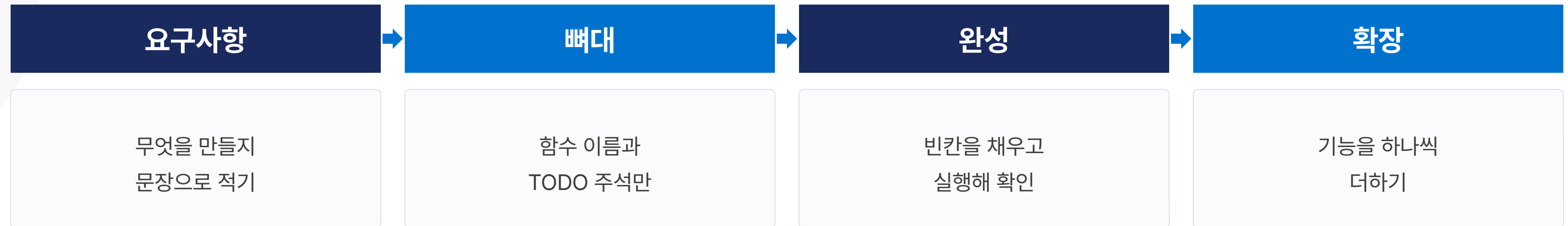
- ① with open 으로 파일을 쓰고 읽는다
- ② r · w · a 모드의 차이를 말한다
- ③ encoding="utf-8" 을 빠뜨리지 않는다
- ④ try · except 로 ValueError 를 잡아 다시 묻는다

PART 6

프로젝트 3종

계산기 · 숫자 맞추기 · 타이머 — 배운 문법을 엮어 실제로 동작하는 프로그램을 만듭니다

프로젝트는 이 순서로



한 번에 다 쓰지 않는다 — 조금 쓰고 실행하고, 또 조금 쓰고 실행한다

요구사항

- ① 연산자(+ - * /)와 숫자 두 개를 입력받는다
- ② 결과를 출력한다 — 0으로 나누면 안내 문구
- ③ 숫자가 아닌 입력은 다시 묻는다
- ④ 모르는 연산자는 안내하고, q 를 넣으면 끝낸다

쓰는 문법 — 함수 · if · while · try/except

계산기 — 뼈대

```
1  def calculate(a, op, b):
2      # TODO ① op 에 따라 + - * / 결과를 돌려준다
3      # TODO ② 0으로 나누면 None 을 돌려준다
4      pass
5
6  def read_number(prompt):
7      # TODO ③ 숫자를 받을 때까지 다시 묻는다
8      pass
9
10 def main():
11     # TODO ④ q 가 나올 때까지 입력 → 계산 → 출력을 반복한다
12     pass
13
14 main()
```

pass 는 「아직 비워 둬」 — 뼈대도 실행은 된다(아무것도 출력하지 않을 뿐)

계산기 — 완성 (1/2)

```
1 def calculate(a, op, b):
2     if op == "+":
3         return a + b
4     elif op == "-":
5         return a - b
6     elif op == "*":
7         return a * b
8     elif op == "/":
9         if b == 0:
10            return None          # 0으로 나누기는 막는다
11            return a / b
12
13 def read_number(prompt):
14     while True:
15         try:
16             return float(input(prompt))
17         except ValueError:
18             print("숫자를 입력하세요.")
```

read_number 는 숫자를 받을 때까지 while True 로 되묻는다

계산기 — 완성 (2/2)

```
19 def main():
20     print("계산기 (끝내려면 연산자에 q)")
21     while True:
22         op = input("연산자(+ - * /): ")
23         if op == "q":
24             print("종료합니다.")
25             break
26         if op not in ["+", "-", "*", "/"]:
27             print("지원하지 않는 연산자입니다.")
28             continue
29         a = read_number("첫 번째 수: ")
30         b = read_number("두 번째 수: ")
31         result = calculate(a, op, b)
32         if result is None:
33             print("0으로 나눌 수 없습니다.")
34         else:
35             print(f"결과: {a} {op} {b} = {result}")
36
37     main()
```

op not in [...] — 목록에 없는 연산자는 **continue** 로 처음부터 다시

계산기 — 실행 예

```
계산기 (끝내려면 연산자에 q)  
연산자(+ - * /): +  
첫 번째 수: 3  
두 번째 수: 4  
결과: 3.0 + 4.0 = 7.0  
연산자(+ - * /): /  
첫 번째 수: 10  
두 번째 수: 0  
0으로 나눌 수 없습니다.  
연산자(+ - * /): %  
지원하지 않는 연산자입니다.  
연산자(+ - * /): *  
첫 번째 수: 둘  
숫자를 입력하세요.  
첫 번째 수: 2  
두 번째 수: 5  
결과: 2.0 * 5.0 = 10.0  
연산자(+ - * /): q  
종료합니다.
```

0으로 나누기 · 모르는 연산자(%) · 숫자가 아닌 입력(둘)까지 넣어 본 실제 실행 화면

확장 과제

연산 더하기

01

거듭제곱 ** 와 나머지 % 를
calculate 에 추가

깔끔한 표시

02

7.0 처럼 정수로 떨어지면
7 로 보여 주기

계산 기록

03

결과를 리스트에 모아
끝날 때 한꺼번에 출력

파일 저장

04

기록을 history.txt 에
이어 쓰기("a" 모드)

요구사항

- ① 컴퓨터가 1~100 사이 숫자를 몰래 정한다
- ② 추측을 입력하면 「더 큰 수」「더 작은 수」를 알려 준다
- ③ 맞으면 시도 횟수를 알려 주고 끝낸다
- ④ 숫자가 아닌 입력은 횟수에 넣지 않고 다시 묻는다

쓰는 문법 — random 모듈 · while True · break · try/except

숫자 맞추기 — 뼈대

```
1 import random
2
3 secret = random.randint(1, 100)    # 컴퓨터가 몰래 정한 수
4 tries = 0
5 while True:
6     # TODO ① 입력을 정수로 바꾼다 — 실패하면 다시 묻는다
7     # TODO ② tries 를 1 늘린다
8     # TODO ③ 크다 · 작다 · 정답을 알려 주고, 정답이면 break
9     break
```

뼈대의 **break** 는 일단 반복을 한 번에 끝내 두는 자리표시 — 완성하면서 정답일 때만 **break** 하게 바꾼다

숫자 맞추기 — 완성

```
1 import random
2
3 secret = random.randint(1, 100)    # 컴퓨터가 몰래 정한 수
4 tries = 0
5 while True:
6     try:
7         guess = int(input("추측(1~100): "))
8     except ValueError:
9         print("정수를 입력하세요.")
10        continue
11    tries += 1
12    if guess < secret:
13        print("더 큰 수입니다.")
14    elif guess > secret:
15        print("더 작은 수입니다.")
16    else:
17        print(f"정답! {tries}번 만에 맞했습니다.")
18        break
```

`random.randint(1, 100)` 은 1과 100을 모두 포함한다 — 실행할 때마다 다른 수가 정해진다

숫자 맞추기 — 실행 예

```
추측(1~100): 50
더 작은 수입니다.
추측(1~100): 오십
정수를 입력하세요.
추측(1~100): 25
더 작은 수입니다.
추측(1~100): 12
더 큰 수입니다.
추측(1~100): 18
더 작은 수입니다.
추측(1~100): 15
더 큰 수입니다.
추측(1~100): 16
정답! 6번 만에 맞혔습니다.
```

절반씩 좁혀 가면(이분 탐색) 1~100 은 7번 안에 반드시 맞힌다 — 숫자가 아닌 입력은 횃수에서 빠진다

확장 과제

범위 검사

01

1~100 밖의 수는
안내하고 다시 묻기

기회 제한

02

7번 안에 못 맞히면
정답을 알려 주고 끝

다시 하기

03

끝난 뒤 「한 판 더?(y/n)」
— 함수로 묶어 반복

난이도

04

쉬움 1~50 · 어려움 1~1000
을 골라 시작

요구사항

- ① 몇 초를 셀지 입력받는다 — 1 이상의 정수
- ② 1초마다 남은 시간을 MM:SS 로 같은 줄에 갱신한다
- ③ 0이 되면 「시간 종료!」를 출력한다
- ④ 숫자가 아니거나 0 이하이면 안내하고 다시 묻는다

쓰는 문법 — time 모듈 · divmod · f-string 서식 · 예외 처리

타이머 — 빼대

```
1 import time
2
3 def countdown(seconds):
4     # TODO ① seconds 가 0보다 큰 동안 반복한다
5     # TODO ② divmod 로 분 · 초를 나눠 MM:SS 로 같은 줄에 출력
6     # TODO ③ 1초 쉬고 seconds 를 1 줄인다
7     print("시간 종료!")
8
9     # TODO ④ 1 이상의 정수를 받을 때까지 묻는다
10    sec = 3
11    countdown(sec)
```

입력 부분을 만들기 전에는 `sec = 3` 처럼 값을 박아 두고 `countdown` 부터 완성한다

타이머 — 완성

```
1 import time
2
3 def countdown(seconds):
4     while seconds > 0:
5         mins, secs = divmod(seconds, 60)      # 몫=분, 나머지=초
6         print(f"{mins:02d}:{secs:02d}", end="\r")  # 같은 줄에 덮어쓰기
7         time.sleep(1)                          # 1초 쉬기
8         seconds -= 1
9         print("시간 종료!")
10
11 while True:
12     try:
13         sec = int(input("몇 초? "))
14         if sec >= 1:
15             break
16         print("1 이상을 입력하세요.")
17     except ValueError:
18         print("정수를 입력하세요.")
19     countdown(sec)
```

`end="\r"` 는 줄을 바꾸지 않고 줄 맨 앞으로 돌아가 같은 자리에 덮어쓴다

타이머 — 실행 예

```
몇 초? 삼
정수를 입력하세요.
몇 초? 0
1 이상을 입력하세요.
몇 초? 3
00:03
00:02
00:01
시간 종료!
```

실제 화면에서는 00:03 → 00:02 → 00:01 이 한 줄에서 바뀐다 — 여기서는 덮어쓴 값을 줄마다 펼쳐 보였다

확장 과제

분:초 입력

01

"1:30" 을 split(":") 으로
나눠 90초로 바꾸기

중단 처리

02

Ctrl+C 를 누르면
KeyboardInterrupt 로 잡아 안내

뽀모도로

03

25분 집중 · 5분 휴식을
정한 횟수만큼 반복

기록

04

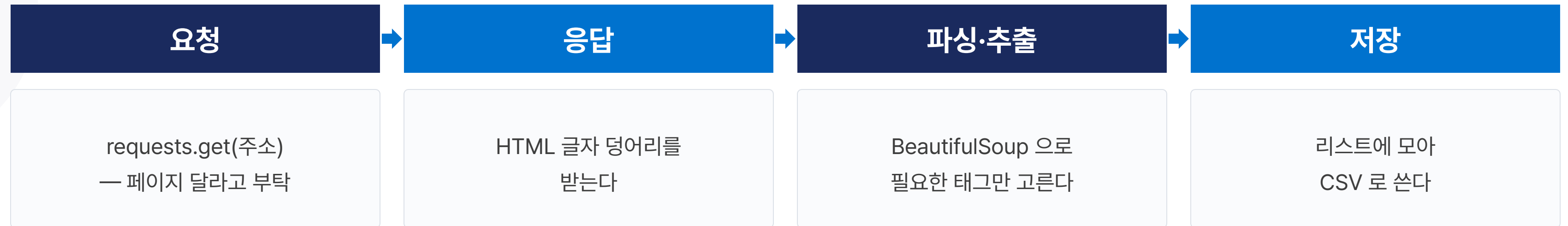
끝난 시각을 datetime 으로
log.txt 에 이어 쓰기

PART 7

웹 크롤링 맛보기

웹 페이지를 받아 오고(requests), 필요한 부분만 골라(BeautifulSoup), 파일로 저장합니다

크롤링의 네 단계



준비 — 가상환경을 켜고 `pip install requests beautifulsoup4`

먼저 지킬 것

robots.txt 와 이용약관

robots.txt	사이트주소/robots.txt 의 허용·금지 경로를 먼저 확인한다
이용약관	자동 수집을 금지하는 조항이 있으면 하지 않는다
서버 배려	요청 사이에 1초 이상 쉬고, 필요한 페이지만 받는다
우회 금지	막아 둔 곳을 브라우저인 척 꾸며서 뚫지 않는다
데이터 취급	개인정보·저작물은 함부로 저장·공개하지 않는다
연습 장소	수집 연습용으로 공개된 사이트에서 연습한다

이 장의 실습 사이트 **books.toscrape.com · quotes.toscrape.com** 은 수집 연습용으로 만든 공개 사이트다

robots.txt 를 코드로 확인하기

```
1 from urllib import robotparser
2
3 # ① 규칙 읽는 법 — 예시 규칙으로 연습
4 rp = robotparser.RobotFileParser()
5 rp.parse(["User-agent: *", "Disallow: /private/"])
6 print(rp.can_fetch("*", "/books/1.html"))
7 print(rp.can_fetch("*", "/private/a.html"))
8
9 # ② 연습 사이트의 robots.txt 확인
10 site = "https://books.toscrape.com"
11 rp2 = robotparser.RobotFileParser(site + "/robots.txt")
12 rp2.read()
13 print(rp2.can_fetch("*", site + "/catalogue/page-2.html"))
14
15 # [실행 결과]
16 # True
17 # False
18 # True
```

연습 사이트는 robots.txt 가 없어(404) 「모두 허용」으로 판정된다 — 결과가 False 인 주소는 수집하지 않는다

책 한 권은 이렇게 생겼다

```
1 <article class="product_pod">
2   <h3>
3     <a href="catalogue/a-light-in-the-attic_1000/index.html"
4       title="A Light in the Attic">A Light in the ...</a>
5   </h3>
6   <div class="product_price">
7     <p class="price_color">£51.77</p>
8   </div>
9 </article>
```

실제 페이지에서 필요한 부분만 줄인 것 — 브라우저에서 F12(개발자 도구) → 요소 고르기로 태그와 class 를 찾는다

requests — 페이지 받아 오기

```
1 import requests
2
3 url = "https://books.toscrape.com/"
4 res = requests.get(url, timeout=10) # 10초 안에 답이 없으면 포기
5 res.raise_for_status()             # 200번대가 아니면 오류로 멈춘다
6 res.encoding = "utf-8"            # 이 사이트의 글자 인코딩
7
8 print(res.status_code)
9 print(res.headers["Content-Type"])
10 print(res.text.strip()[:15])
11
12 # [실행 결과]
13 # 200
14 # text/html
15 # <!DOCTYPE html>
```

상태 코드 200 은 성공, 404 는 없는 페이지, 500 은 서버 오류 — `raise_for_status()` 가 실패를 바로 알려 준다

BeautifulSoup — 태그 고르기

```
1 from bs4 import BeautifulSoup
2 html = """
3 <ul id="menu">
4   <li class="item">김밥 <span class="price">3000</span></li>
5   <li class="item">라면 <span class="price">4500</span></li>
6 </ul>
7 """
8 soup = BeautifulSoup(html, "html.parser")
9
10 print(soup.find("li").get_text(" ", strip=True)) # 첫 번째 하나
11 print(len(soup.find_all("li")))                 # 전부 — 리스트
12 for p in soup.select("li.item .price"):          # CSS 선택자
13     print(p.get_text())
14
15 # [실행 결과]
16 # 김밥 3000
17 # 2
18 # 3000
19 # 4500
```

find 는 첫 하나, find_all · select 는 전부(리스트) — select 는 CSS 선택자(.class · #id · 태그) · 속성 값은 태그["이름"]

첫 페이지의 책 제목과 가격

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 res = requests.get("https://books.toscrape.com/", timeout=10)
5 res.encoding = "utf-8"
6 soup = BeautifulSoup(res.text, "html.parser")
7
8 books = soup.select("article.product_pod")
9 print(len(books), "권")
10 for book in books[:3]:
11     title = book.h3.a["title"]          # 전체 제목은 title 속성에
12     price = book.select_one(".price_color").get_text()
13     print(price, title)
14
15 # [실행 결과]
16 # 20 권
17 # £51.77 A Light in the Attic
18 # £53.74 Tipping the Velvet
19 # £50.10 Soumission
```

화면에 보이는 제목은 「...」로 잘려 있다 — 전체 제목은 a 태그의 title 속성에 들어 있다

여러 페이지 모아 CSV 로 저장

```
1 import csv, time
2 import requests
3 from bs4 import BeautifulSoup
4 URL = "https://books.toscrape.com/catalogue/page-{}.html"
5 rows = []
6 for page in range(1, 4):          # 1~3쪽만
7     res = requests.get(URL.format(page), timeout=10)
8     res.encoding = "utf-8"
9     soup = BeautifulSoup(res.text, "html.parser")
10    for book in soup.select("article.product_pod"):
11        price = book.select_one(".price_color").get_text()
12        rows.append([book.h3.a["title"], price])
13    time.sleep(1)                  # 서버 배려 — 1초 쉬기
14 with open("books.csv", "w", newline="", encoding="utf-8-sig") as f:
15     csv.writer(f).writerows([["제목", "가격"]] + rows)
16 print(len(rows), "권 저장")
17
18 # [실행 결과]
19 # 60 권 저장
```

페이지 주소의 숫자만 바꿔 가며 요청한다 — 엑셀에서 열 파일이라 utf-8-sig 로 저장했다

requests 로 안 보이는 페이지도 있다

정적 페이지

quotes.toscrape.com

- 서버가 내용이 다 담긴 HTML 을 보낸다
- requests 로 받은 HTML 에 내용이 있다
- 이 장의 방법으로 수집된다

동적 페이지

quotes.toscrape.com/js/

VS

- 빈 틀을 받은 뒤 JavaScript 가 내용을 채운다
- requests 로는 내용이 비어 보인다
- 브라우저를 움직이는 도구가 필요하다

같은 내용, 다른 결과 — 직접 확인

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 for url in ["https://quotes.toscrape.com/",
5             "https://quotes.toscrape.com/js/"]:
6     res = requests.get(url, timeout=10)
7     soup = BeautifulSoup(res.text, "html.parser")
8     print(url, "→ 명언", len(soup.select("div.quote")), "개")
9
10 # [실행 결과]
11 # https://quotes.toscrape.com/ → 명언 10 개
12 # https://quotes.toscrape.com/js/ → 명언 0 개
```

동적 페이지는 Selenium · Playwright 처럼 브라우저를 직접 움직이는 도구로 다루거나, 사이트가 공개한 API 를 쓴다

흔한 오류 ⑦ 아무것도 못 찾았다

```
1 import requests
2 from bs4 import BeautifulSoup
3
4 res = requests.get("https://books.toscrape.com/", timeout=10)
5 soup = BeautifulSoup(res.text, "html.parser")
6
7 tag = soup.select_one(".book-title") # 없는 class 이름(오타)
8 print(tag)
9 print(tag.get_text())
10
11 # [실행 결과 — 오류]
12 # None
13 # AttributeError: 'NoneType' object has no attribute 'get_text'
```

선택자가 아무것도 못 찾으면 None — 개발자 도구로 class 이름을 다시 보고, if tag: 로 먼저 검사한다

끝까지 왔다면 확인할 것

환경	venv 를 만들어 켜고 pip 로 설치할 수 있다
문법	변수·조건·반복으로 작은 문제를 푼다
자료구조	리스트와 딕셔너리를 골라 쓰고 순회한다
함수·모듈	return 하는 함수를 만들고 import 한다
파일·예외	utf-8 로 읽고 쓰며 try·except 로 지킨다
프로젝트	요구사항 → 뼈대 → 완성 → 확장으로 만든다
크롤링	robots.txt·약관을 확인하고 정적 페이지를 모은다

다음 학습

여기서 어디로 갈까

데이터 분석

01

pandas 로 CSV 를 표처럼 다루고
matplotlib 로 그래프 그리기

자동화

02

폴더 정리 · 엑셀 보고서 · 메일 같은
반복 업무를 스크립트로

동적 페이지

03

Selenium · Playwright 로
브라우저를 움직여 수집

객체지향

04

클래스 · 상속으로
큰 프로그램을 나눠 설계

H01 마무리

파이썬 기초·응용·심화 마무리

문법은 외우는 것이 아니라 쳐 보고 틀려 보며 손에 익히는 것입니다

✓ 가상환경을 만들고 VS Code 에서 실행

✓ 파일·예외로 튼튼한 프로그램

✓ 변수·조건·반복·자료구조·함수

✓ 규칙을 지켜 웹 데이터 수집

오늘의 산출물

프로젝트 3개 + books.csv

CREDITS · CONTACT

출처 · 문의

이애본 (Aebon Lee, Ph.D) — 드림아이티비즈 대표 · 한신대학교 AI·SW대학 겸임교수

useful-ai.dreamitbiz.com — 강의안·영상 강의 라이브러리

aebon@dreamitbiz.com · 카카오톡 ID aebon

© 2026 이애본 · 발행 홍문원(洪文院) · CC BY-NC-ND 4.0